

Spécification et preuve de programmes

Solution du devoir 2 : spécifications et preuves avec micro-C

Alain Giorgetti

Licence d'Informatique de l'Université de Franche-Comté 2023-24

Ce devoir doit être rendu sous la forme d'un unique fichier texte *NomPrenom.c*, où *NomPrenom* sont les 8 premières lettres de votre nom de famille, suivies des 8 premières lettres de votre prénom, sans espaces ni accents. Ce fichier doit commencer par un commentaire entre */** et **/* indiquant vos nom et prénom. Ensuite, des commentaires entre */** et **/* doivent indiquer le numéro de chaque question traitée, par exemple

```
/* Exercice 2 question 3 */
```

juste avant la réponse à la question 3 de l'exercice 2.

La preuve et l'exécution du contenu complet de ce fichier doivent être possibles avec l'interface de micro-C en ligne. Dans ce but, les parties incorrectes ou incomplètes doivent être entourées par */** et **/*.

Lire l'énoncé jusqu'à la fin avant de commencer à traiter les exercices.

1 Propriétés des tableaux d'entiers micro-C (5 points)

L'objectif de cet exercice est d'écrire des formules micro-C qui formalise des propriétés sur les tableaux d'entiers C.

1. Reproduire et compléter la définition

```
predicate cte(int a[], int c) =
```

pour définir en micro-C la propriété que tous les éléments du tableau *a* ont la même valeur *c*.

(1 point)

```
predicate cte(int a[], int c) = forall i. 0 <= i < length(a) -> a[i] == c;
```

2. De même, définir en micro-C, par un prédicat `no_dup(int a[])`, la propriété que toutes les valeurs du tableau *a* sont deux à deux distinctes.

(1 point)

```
predicate no_dup(int a[]) = forall i,j. 0 <= i < j < length(a) -> a[i] != a[j];
```

3. Utiliser ces deux prédicats pour formaliser en micro-C le lemme suivant : "Tout tableau constant qui a au moins deux éléments n'a pas tous ses éléments deux à deux distincts."

(1 point)

```
lemma cte_no_dup: forall a[]. length(a) > 1 -> forall c. cte(a,c) -> !no_dup(a);
```

4. Même si ce lemme est vrai et bien écrit en micro-C, vous pouvez observer que l'interface de micro-C ne permet pas de le démontrer, car le prouveur ne sait pas bien choisir des indices particuliers dans le tableau comme témoins. Pour aider le prouveur, formalisez les lemmes suivants :

- (a) “Dans tout tableau t constant qui a au moins deux éléments, les cases $t[0]$ et $t[1]$ sont égales”, et
- (b) “Tout tableau t qui a au moins deux éléments et dont les cases $t[0]$ et $t[1]$ sont égales a des éléments non deux à deux distincts.”

(1 point)

```
lemma cte01: forall t[]. (exists c. cte(t,c)) && length(t) > 1 -> t[0] == t[1];  
  
lemma eq01_no_dup: forall t[]. length(t) > 1 && t[0] == t[1] -> !no_dup(t);
```

5. Où doivent être placés ces lemmes pour aider la démonstration du lemme de la question 3 ?

(1 point) Pour permettre la preuve du lemme `cte_no_dup`, les lemmes `cte01` et `eq01_no_dup` doivent être placés avant.

```
/* Exercice 1 questions 1 - 4 */  
  
/*@  
  predicate cte(int a[], int c) = forall i. 0 <= i < length(a) -> a[i] == c;  
  
  predicate no_dup(int a[]) = forall i,j. 0 <= i < j < length(a) -> a[i] != a[j];  
  
  lemma cte_no_dup_not_proved:  
    forall a[]. length(a) > 1 -> forall c. cte(a,c) -> !no_dup(a);  
  
  lemma cte01: forall t[]. (exists c. cte(t,c)) && length(t) > 1 -> t[0] == t[1];  
  
  lemma eq01_no_dup: forall t[]. length(t) > 1 && t[0] == t[1] -> !no_dup(t);  
  
  lemma cte_no_dup: forall a[]. length(a) > 1 -> forall c. cte(a,c) -> !no_dup(a);  
*/  
  
/* Exercice 1 question 5 : Pour permettre la preuve du lemme cte_no_dup, les deux  
   lemmes cte01 et eq01_no_dup doivent etre avant lui dans le fichier. */
```

Listing 1: Réponses de l'exercice 1 dans le fichier `NomPrenom.c`.

2 Calcul du carré d'un nombre entier (4 points)

1. Définir une fonction C calculant le carré de son paramètre entier x , selon l'algorithme suivant :

$i := 0; c := 0; \text{ while } i < x \text{ do } c := c + 2i + 1; i := i + 1 \text{ od}$

```
/* Exercice 2 question 1, 1 point : code C sans annotations. */
int square1 (int x) {
    int i = 0;
    int c = 0;
    while (i < x) {
        c = c + 2 * i + 1;
        i = i+1;
    }
    return(c);
}
```

2. Spécifier le contrat de cette fonction en micro-C.

```
/*@ requires x >= 0;                /* Q2, 0.5 pt */
    ensures result == x * x; */    /* Q2, 0.5 pt */
```

3. Spécifier un invariant de boucle assez précis pour permettre de démontrer la postcondition de ce contrat.

```
invariant c == i * i && i <= x; /* Q3, 1 point */
```

4. Spécifier un variant de boucle permettant de démontrer la terminaison de cette boucle.

```
variant x - i;                /* Q4, 1 point */
```

5. Prouver le contrat avec l'interface en ligne de micro-C.

(pas de point sur cette question)
Tout est prouvé automatiquement dans l'interface en ligne de micro-C.

3 Calcul du maximum (4 points)

1. Définir en micro-C une fonction

```
int max (int t[], int n)
```

qui calcule l'indice d'un élément maximal dans tout tableau d'entiers `t` de longueur `n` non nulle.

```
/* Exercice 3 question 1, 0.5 point : code C sans annotations. */
int max (int t[], int n) {
    int imax = 0;
    int k = 0;
    while (k < n) {
        if (t[imax] < t[k]) imax = k;
        k = k + 1;
    }
    return(imax);
}
```

2. Spécifier le contrat de cette fonction en micro-C.

```
/*@ requires 0 < n == length(t);                /* Q2, 0.5 pt */
    ensures 0 <= result < n;                    /* Q2, 0.5 pt */
    ensures forall j. 0 <= j < n -> t[result] >= t[j]; /* Q2, 0.5 pt */
```

3. Ajouter des invariants et un variant de boucle pour prouver ce contrat et la terminaison de la fonction.

```
/*@ invariant 0 <= imax <= k;
    invariant imax < n;                /* Q3, 0.5 pt */
    invariant 0 <= k <= n;            /* Q3, 0.5 pt */
    invariant forall j. 0 <= j < k -> t[imax] >= t[j]; /* Q3, 0.5 pt */
    variant n - k; /*
```

4. Vérifier que la preuve est automatique avec l'interface en ligne de micro-C.

(pas de point sur cette question)

Tout est prouvé automatiquement dans l'interface en ligne de micro-C.

4 Recherche dichotomique en micro-C (7 points)

Cet exercice est la suite de l'exercice 3 du devoir 1.

1. Implémenter en C le programme de recherche dichotomique de la figure 2.4 du cours.

```
/* Exercice 4 question 1, 1 point : code C sans annotations. */
int bin_search(int t[], int n, int x) {
    int posx = -1;
    int min = -1;
    int max = n-1;
    while (min != max) {
        int m = (min + max + 1) / 2;
        if (t[m] <= x)
            min = m;
        else
            max = m-1;
    }
    posx = min+1;
    return posx;
}
```

2. Spécifier le contrat de cette fonction en micro-C, en utilisant les prédicats définis dans le devoir 1.

```
/* Exercice 4, question 2 */
/*@ predicate posx_post(int posx, int n) = 0 <= posx <= n;
    predicate pre(int t[], int n) =
        0 <= n <= length(t) && forall i, j. 0 <= i < j < n -> t[i] <= t[j];
    predicate post(int t[], int n, int x, int posx) =
        forall k. (0 <= k < posx -> t[k] <= x) && (posx <= k < n -> t[k] > x); */
int bin_searchComplete(int t[], int n, int x) {
    /*@ requires pre(t,n);          /* Q2, 1 pt */
    ensures posx_post(result,n);    /* Q2, 1 pt */
    ensures post(t,n,x,result);    /* Q2, 1 pt */
}
```

3. Ajouter des invariants et un variant de boucle pour prouver ce contrat et la terminaison de la fonction.

```
/*@ invariant -1 <= min <= max < n;          /* Q3, 0.5 pt */
    invariant forall j. 0 <= j <= min -> t[j] <= x; /* Q3, 1 pt */
    invariant forall j. max < j < n -> t[j] > x; /* Q3, 1 pt */
    variant max - min; /* Q3, 0.5 pt */
```

4. Vérifier que la preuve est automatique avec l'interface en ligne de micro-C.

(pas de point sur cette question)

Tout est prouvé automatiquement dans l'interface en ligne de micro-C.